

ROLLBACK

ROLLBACK is used to undo a transaction block. See Chapter 26 for more information.

► Input

```
| ROLLBACK [ TO savepointname];
```

SAVEPOINT

SAVEPOINT defines a savepoint for use with a ROLLBACK statement. See Chapter 26 for more information.

► Input

```
| SAVEPOINT sp1
```

SELECT

SELECT is used to retrieve data from one or more tables (or views). See Chapter 4, “Retrieving Data,” Chapter 5, “Sorting Retrieved Data,” and Chapter 6, “Filtering Data,” for more basic information. (Chapters 4–17 also cover aspects of SELECT.)

► Input

```
| SELECT columnname, ...  
| FROM tablename, ...  
| [WHERE ...]  
| [UNION ...]  
| [GROUP BY ...]  
| [HAVING ...]  
| [ORDER BY ...];  
| ;
```

START TRANSACTION

START TRANSACTION is used to start a new transaction block. See Chapter 26 for more information.

► Input

```
| START TRANSACTION;
```

UPDATE

UPDATE updates one or more rows in a table. See Chapter 20 for more information.

► Input

```
| UPDATE tablename  
| SET columnname = value, ...  
| [WHERE ...];
```

MySQL Datatypes

This appendix explains the different datatypes used in MySQL.

As explained in Chapter 1, “Understanding SQL,” datatypes are basically rules that define what data may be stored in a column and how that data is actually stored.

Datatypes are used for several reasons:

- Datatypes enable you to restrict the type of data that can be stored in a column. For example, a numeric datatype column accepts only numeric values.
- Datatypes allow for more efficient storage internally. For example, numbers and datetime values can be stored in a more condensed format than can text strings.
- Datatypes allow for alternate sorting orders. If everything is treated as strings, 1 comes before 10, which comes before 2. (Strings are sorted in dictionary sequence, one character at a time starting from the left.) As numeric datatypes, the numbers would be sorted correctly.

When designing tables, pay careful attention to the datatypes being used. Using the wrong datatype can seriously impact your application. Changing the datatypes of existing populated columns is not a trivial task. (In addition, doing so can result in data loss.)

Although this appendix is by no means a complete tutorial on datatypes and how they are to be used, it explains the major MySQL datatype types and what they are used for.

String Datatypes

The most commonly used datatype is the string datatype. This datatype stores a string, such as names, addresses, phone numbers, or zip codes. As listed in Table D.1, there are basically two types of string datatype that you can use: fixed-length strings and variable-length strings.

A fixed-length string is a datatype that is defined to accept a fixed number of characters, and that number is specified when the table is created. For example, you might allow 30 characters in a first name column or 11 characters in a Social Security number column (which is the exact number needed plus the two dashes). Fixed-length columns do not allow more than the specified number of characters. They also allocate storage space for as many characters as specified. So, if the string **Ben** is stored in a 30-character first name field, a full 30 bytes are stored. **CHAR** is an example of a fixed-length string type.

Variable-length strings store text of variable length. Some variable-length datatypes have a defined maximum size. Others are entirely variable. Either way, only the data specified is saved (and no extra data is stored). TEXT is an example of a variable-length string type.

If variable-length datatypes are so flexible, why would you ever want to use fixed-length datatypes? The answer is performance. MySQL can sort and manipulate fixed-length columns far more quickly than it can sort variable-length columns. In addition, MySQL does not allow you to index variable-length columns (or the variable portion of a column). This also dramatically affects performance.

Table D.1 String Datatypes

Datatype	Description
CHAR	Fixed-length string from 1 to 255 characters long. Its size must be specified at creation time, or MySQL assumes CHAR(1).
ENUM	Accepts one of a predefined set of up to 64KB strings.
LONGTEXT	Same as TEXT, but with a maximum size of 4GB.
MEDIUMTEXT	Same as TEXT, but with a maximum size of 16KB.
SET	Accepts zero or more of a predefined set of up to 64 strings.
TEXT	Variable-length text with a maximum size of 64KB.
TINYTEXT	Same as TEXT, but with a maximum size of 255 bytes.
VARCHAR	Same as CHAR, but stores just the text. The size is a maximum, not a minimum.

Tip

Use Quotation Marks Regardless of the form of string datatype being used, string values must always be surrounded by quotation marks. (Single quotation marks are often preferred.)

Caution

When Numeric Values Are Not Numeric Values You might think that phone numbers and zip codes should be stored in numeric fields (after all, they store only numeric data), but doing so would not be advisable. If you store the zip code 01234 in a numeric field, the number 1234 will be saved, and you'll actually lose a digit.

The basic rule to follow is this: If the number is a number used in calculations (sums, averages, and so on), it belongs in a numeric datatype column. If it is used as a literal string (that happens to contain only digits), it belongs in a string datatype column.

Numeric Datatypes

Numeric datatypes store numbers. MySQL supports several numeric datatypes, each with a different range of numbers that can be stored in it. Obviously, the larger the supported range, the more storage space needed. In addition, some numeric datatypes support the use of decimal points (and fractional numbers), whereas others support only whole numbers. Table D.2 lists the frequently used MySQL numeric datatypes.

Note

Signed or UNSIGNED? All numeric datatypes, with the exception of **BIT** and **BOOLEAN**, can be signed or unsigned. Signed numeric columns can store both positive and negative numbers, and unsigned numeric columns store only positive numbers. Signed is the default, but if you know that you'll not need to store negative values, you can use the **UNSIGNED** keyword, which will allow you to store values twice as large.

Table D.2 Numeric Datatypes

Datatype	Description
BIGINT	Integer value that supports numbers from -9223372036854775808 to 9223372036854775807 (or 0 to 18446744073709551615 if UNSIGNED).
BIT	Bit field from 1 to 64 bits wide. (Prior to MySQL 5, BIT was functionally equivalent to TINYINT.)
BOOLEAN (or BOOL)	Boolean flag, either 0 or 1, used primarily for on/off flags.
DECIMAL (or DEC)	Floating point value with varying level of precision.
DOUBLE	Double-precision floating point value.
FLOAT	Single-precision floating point value.
INT (or INTEGER)	Integer value that supports numbers from -2147483648 to 2147483647 (or 0 to 4294967295 if UNSIGNED).
MEDIUMINT	Integer value that supports numbers from -8388608 to 8388607 (or 0 to 16777215 if UNSIGNED).
REAL	4-byte floating point value.
SMALLINT	Integer value that supports numbers from -32768 to 32767 (or 0 to 65535 if UNSIGNED).
TINYINT	Integer value that supports numbers from -128 to 127 (or 0 to 255 if UNSIGNED).

Tip

Don't Use Quotation Marks Unlike strings, numeric values should never be enclosed within quotes.

Tip

Storing Currency There is no special MySQL datatype for currency values. Use `DECIMAL(8,2)` for currency.

Date and Time Datatypes

MySQL uses special datatypes for the storage of date and time values, as listed in Table D.3.

Table D.3 Date and Time Datatypes

Datatype	Description
DATE	Date from 1000-01-01 to 9999-12-31 in the format <code>YYYY-MM-DD</code> .
DATETIME	A combination of DATE and TIME.
TIMESTAMP	Functionally equivalent to DATETIME (but with a smaller range).
TIME	Time in the format <code>HH:MM:SS</code> .
YEAR	A two- or four-digit year; two-digit years support the range 70 (1970) to 69 (2069), and four-digit years support the range 1901 to 2155.

Binary Datatypes

Binary datatypes are used to store all sorts of data (even binary information), such as graphic images, multimedia, and word processor documents (see Table D.4).

Table D.4 Binary Datatypes

Datatype	Description
BLOB	Blob with a maximum length of 64KB.
MEDIUMBLOB	Blob with a maximum length of 16MB.
LONGBLOB	Blob with a maximum length of 4GB.
TINYBLOB	Blob with a maximum length of 255 bytes.

Note

Datatypes in Use If you would like to see a real-world example of how different datatypes are used, see the sample table creation scripts described in Appendix B, “The Example Tables.”